

Punto 3.3 — Proyecciones con Select y tipos anónimos

Contexto del proyecto

Este checkpoint continúa M03/PROYECTO/3.2. Se mantiene la separación Domain/Application/Infraestructure/Console y el esquema se obtiene con migraciones, no con `EnsureCreated()`.

Objetivo práctico

Implementar y ejecutar **Proyecciones con Select y tipos anónimos** sobre `AceriaData`, inspeccionando resultados reales y, cuando corresponde, el SQL generado por EF Core 8.

Paso 1: Abrir el checkpoint

```
cd M03/PROYECTO/3.3
dotnet restore AceriaData.sln
```

La solución contiene los cuatro proyectos y puede abrirse directamente en Visual Studio 2022/2026 compatible con .NET 8.

Paso 2: Comprobar la historia de migraciones

```
dotnet ef migrations list --project src/AceriaData.Infraestructure --startup-project src/AceriaData.Console
```

El listado debe conservar las migraciones de M1 y M2, incluida `M2_2_12_Architecture`. LINQ y las estrategias de carga de M3 no cambian el esquema, por lo que M3 no inventa migraciones vacías.

Paso 3: Revisar los cambios de este punto

Los elementos trazados en este estado son: `ObtenerClientesUnicos`, `ObtenerResumenes`, `ObtenerResumenesPorEstado`, `ObtenerOrdenesConTotales`, `ObtenerSqlProyeccion`. El contrato crece de forma acumulativa, salvo en 3.12, donde el seam docente `IQueryable` se retira del puerto público y la composición queda encapsulada en `Infraestructure`.

Paso 4: Implementar y estudiar el caso de uso

Archivo: `src/AceriaData.Application/ProyeccionesUseCase.cs`

```

using AceriaData.Application.Interfaces;

namespace AceriaData.Application.UseCases;

public sealed class ProyeccionesUseCase
{
    private readonly IUnidadDeTrabajo _unidad;
    public ProyeccionesUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;

    public void Ejecutar()
    {
        Console.WriteLine("=== PROYECCIONES CON SELECT ===");
        var clientes = _unidad.Ordenes.ObtenerClientesUnicos();
        var resúmenes = _unidad.Ordenes.ObtenerResúmenes();
        var pendientes = _unidad.Ordenes.ObtenerResúmenesPorEstado("Pendiente");
        var totales = _unidad.Ordenes.ObtenerOrdenesConTotales();
        if (clientes.Count != 3 || resúmenes.Count != 5 || pendientes.Count != 3 || totales.Count != 5)
            throw new InvalidOperationException("Proyecciones inesperadas.");
        Console.WriteLine($"Clientes: {string.Join(", ", clientes)} | Resúmenes: {resúmenes.Count}");
        Console.WriteLine(_unidad.Ordenes.ObtenerSqlProyeccion());
    }
}

```

Explicación línea a línea del caso de uso 3.3

Línea 1: `using AceriaData.Application.Interfaces;` → Importa los puertos de Application, en especial `IUnidadDeTrabajo` e `IOrdenRepositorio`, que desacoplan el caso de uso de EF Core.

Línea 3: `namespace AceriaData.Application.UseCases;` → Declara el espacio de nombres de la capa a la que pertenece el archivo.

Línea 5: `public sealed class ProyeccionesUseCase` → Declara el caso de uso concreto que coordina la demostración del punto.

Línea 6: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 7: `private readonly IUnidadDeTrabajo _unidad;` → Guarda el puerto de unidad de trabajo que da acceso al repositorio sin depender de `DbContext`.

Línea 8: `public ProyeccionesUseCase(IUnidadDeTrabajo unidad) => _unidad = unidad;` → Recibe la unidad de trabajo mediante inyección de dependencias.

Línea 10: `public void Ejecutar()` → Define la operación docente que ejecutará el composition root.

Línea 11: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 12: `Console.WriteLine("=== PROYECCIONES CON SELECT ===");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 13: `var clientes = _unidad.Ordenes.ObtenerClientesUnicos();` → Proyecta Cliente y elimina duplicados para comprobar una proyección escalar con `Distinct`.

Línea 14: `var resúmenes = _unidad.Ordenes.ObtenerResúmenes();` → Obtiene la proyección de las cinco órdenes a DTOs de resumen sin materializar entidades completas.

Línea 15: `var pendientes = _unidad.Ordenes.ObtenerResúmenesPorEstado("Pendiente");` → Combina filtro por estado y proyección a DTO para validar que ambas operaciones se ejecutan como una sola consulta.

Línea 16: `var totales = _unidad.Ordenes.ObtenerOrdenesConTotales();` → Proyecta cada orden junto con agregados de sus planchas para validar una proyección calculada sobre navegación.

Línea 17: `if (clientes.Count != 3 || resúmenes.Count != 5 || pendientes.Count != 3 || totales.Count != 5) throw new InvalidOperationException("Proyecciones inesperadas.");` →

Comprueba una condición E2E y hace fallar la ejecución si los resultados no coinciden con los datos esperados.

Línea 18: `Console.WriteLine($"Clientes: {string.Join(", ", clientes)} | Resúmenes: {resumenes.Count}");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 19: `Console.WriteLine(_unidad.Ordenes.ObtenerSqlProyeccion());` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Línea 20: `}` → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Línea 21: `}` → Cierra el bloque sintáctico abierto previamente y termina el ámbito correspondiente.

Paso 5: Preparar y ejecutar el composition root

Archivo: `src/AceriaData.Console/Program.cs`

```
using AceriaData.Application.UseCases;
using AceriaData.ConsoleApp;
using AceriaData.Infrastructure;
using AceriaData.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;

var configuration = new ConfigurationBuilder()
    .SetBasePath(AppContext.BaseDirectory)
    .AddJsonFile("appsettings.json", optional: false)
    .AddEnvironmentVariables()
    .Build();

var connectionString = configuration.GetConnectionString("AceriaDB")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");

var services = new ServiceCollection();
services.AddAceriaInfrastructure(connectionString);
services.AddScoped<ProyeccionesUseCase>();

using var provider = services.BuildServiceProvider(new ServiceProviderOptions
{
    ValidateOnBuild = true,
    ValidateScopes = true
});
using var scope = provider.CreateScope();

var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();
context.Database.EnsureDeleted();
context.Database.Migrate();
DemoData.Seed(context);
context.ChangeTracker.Clear();

var useCase = scope.ServiceProvider.GetRequiredService<ProyeccionesUseCase>();
useCase.Ejecutar();

Console.WriteLine("3.3 OK");
```

Explicación línea a línea de Program.cs 3.3

Línea 1: `using AceriaData.Application.UseCases;` → Importa los casos de uso de Application para poder registrarlos y resolverlos desde el composition root.

Línea 2: `using AceriaData.ConsoleApp;` → Importa DemoData, responsable de sembrar el conjunto determinista usado por las pruebas E2E del checkpoint.

Línea 3: `using AceriaData.Infrastructure;` → Importa la extensión AddAceriaInfrastructure, que registra DbContext, repositorio y unidad de trabajo.

Línea 4: `using AceriaData.Infrastructure.Persistence;` → Importa `AceriaDbContext` para preparar la base de demostración desde el `composition root`.

Línea 5: `using Microsoft.EntityFrameworkCore;` → Importa las extensiones de EF Core utilizadas por `Migrate`, `ChangeTracker` y las operaciones de consulta del `checkpoint`.

Línea 6: `using Microsoft.Extensions.Configuration;` → Importa `ConfigurationBuilder` y las extensiones necesarias para cargar `appsettings.json` y variables de entorno.

Línea 7: `using Microsoft.Extensions.DependencyInjection;` → Importa `ServiceCollection`, `AddScoped`, `CreateScope` y `GetRequiredService` para componer el grafo de dependencias.

Línea 9: `var configuration = new ConfigurationBuilder()` → Inicia la construcción de la configuración de la aplicación.

Línea 10: `.SetBasePath(AppContext.BaseDirectory)` → Fija como base el directorio de ejecución para localizar `appsettings.json`.

Línea 11: `.AddJsonFile("appsettings.json", optional: false)` → Carga la configuración JSON obligatoria que contiene la cadena de conexión.

Línea 12: `.AddEnvironmentVariables()` → Permite sobrescribir configuración mediante variables de entorno.

Línea 13: `.Build();` → Finaliza la construcción y devuelve la configuración inmutable que se usará para leer `ConnectionStrings:AceriaDB`.

Línea 15: `var connectionString = configuration.GetConnectionString("AceriaDB")` → Obtiene la cadena `AceriaDB` desde la configuración.

Línea 16: `?? throw new InvalidOperationException("Falta ConnectionStrings:AceriaDB");` → Detiene el arranque con un error explícito si no existe la cadena de conexión `AceriaDB`, evitando ejecutar el ejercicio con una configuración incompleta.

Línea 18: `var services = new ServiceCollection();` → Crea el contenedor de servicios del `composition root`.

Línea 19: `services.AddAceriaInfrastructure(connectionString);` → Registra `DbContext`, repositorio y unidad de trabajo de `Infrastructure`.

Línea 20: `services.AddScoped<ProyeccionesUseCase>();` → Registra el caso de uso con ciclo de vida `Scoped` para compartir el mismo ámbito que `DbContext`.

Línea 22: `using var provider = services.BuildServiceProvider(new ServiceProviderOptions` → Construye el proveedor de servicios y lo declara con `using` para liberar correctamente los servicios `IDisposable` al terminar el proceso.

Línea 23: `{` → Abre el bloque de la clase, método, inicializador u opción declarada inmediatamente antes; su cierre delimita exactamente ese ámbito.

Línea 24: `ValidateOnBuild = true,` → Obliga al contenedor a comprobar durante la construcción que los servicios registrados pueden resolverse.

Línea 25: `ValidateScopes = true` → Activa la validación de ciclos de vida para detectar, por ejemplo, un servicio `Scoped` consumido incorrectamente desde un `Singleton`.

Línea 26: `});` → Cierra las opciones y completa la llamada que construye el proveedor de servicios.

Línea 27: `using var scope = provider.CreateScope();` → Crea el ámbito `Scoped` que compartirá `AceriaDbContext`, `IUnidadDeTrabajo` y el caso de uso durante esta ejecución.

Línea 29: `var context = scope.ServiceProvider.GetRequiredService<AceriaDbContext>();` → Resuelve el `DbContext` para preparar la base de demostración.

Línea 30: `context.Database.EnsureDeleted();` → Elimina solo la base de demostración para que el E2E comience desde un estado reproducible; no crea el esquema.

Línea 31: `context.Database.Migrate();` → Aplica la historia real de migraciones heredada de M2 para crear o actualizar el esquema.

Línea 32: `DemoData.Seed(context);` → Inserta el conjunto determinista de órdenes, planchas, detalles y aleaciones usado por los E2E.

Línea 33: `context.ChangeTracker.Clear();` → Desacopla las entidades sembradas para que las consultas se materialicen de nuevo desde SQL.

Línea 35: `var useCase = scope.ServiceProvider.GetRequiredService<ProyeccionesUseCase>();` → Resuelve el caso de uso concreto registrado para este checkpoint.

Línea 36: `useCase.Ejecutar();` → Ejecuta la demostración y sus aserciones E2E.

Línea 38: `Console.WriteLine("3.3 OK");` → Escribe una evidencia legible en consola para poder inspeccionar el flujo o el SQL.

Paso 6: Compilar el estado acumulativo

```
dotnet build AceriaData.sln --configuration Release
```

La compilación debe terminar sin errores. Application no referencia Entity Framework Core; todas las llamadas específicas de EF permanecen en Infrastructure o en el composition root.

Paso 7: Ejecutar el E2E contra SQL Server LocalDB

```
dotnet run --project src/AceriaData.Console/AceriaData.Console.csproj --configuration Release
```

El proceso elimina la base de demostración, aplica `Database.Migrate()`, inserta un dataset determinista, limpia el Change Tracker y ejecuta el caso de uso. El marcador final obligatorio es:

```
3.3 OK
```

Paso 8: Analizar resultado y SQL

El caso de uso contiene aserciones que hacen fallar el proceso si cambian los resultados esperados. Cuando el punto estudia traducción SQL, el repositorio devuelve `ToQueryString()` y la consola imprime exactamente la sentencia que EF Core 8 ha construido para SQL Server. No se usa una sentencia SQL ficticia como prueba.

Paso 9: Diagnosticar un error común

Error: Un tipo anónimo no puede actuar como contrato público entre capas; cuando el resultado cruza la frontera se usa un DTO.

Para diagnosticarlo, compara el método del repositorio, la salida E2E y el SQL obtenido con `ToQueryString()` cuando esté disponible. No cambies simultáneamente datos, query y expectativa: modifica una sola variable para poder atribuir la causa.

Código activable para reproducir el error controlado

Estos bloques ya existen comentados en el checkpoint. Actívalos sólo para reproducir esta variante y restaura después el estado original.

Archivo: `src/AceriaData.Application/Interfaces.cs`

Marcador: `ERROR CONTROLADO M03 3.3 - TIPO ANONIMO COMO CONTRATO PUBLICO`

```
// 'var' no puede usarse como tipo de retorno público: el contrato debe usar un DTO con nombre.
var ObtenerResumenAnonimo();
```

Comprobación esperada: un tipo anónimo no puede ser contrato público entre capas.

Laboratorio adicional del punto 3.3

Diagnóstico técnico

Compara materializar la entidad completa con proyectar sólo `NumeroOrden`, `Cliente`, `Estado` y `FechaCreacion`. La ventaja demostrable es la reducción del shape transferido y de entidades materializadas. `Distinct()` debe permanecer en servidor si se aplica antes de `ToList()`.

Reto resuelto y verificación adicional

Usa `ObtenerClientesUnicos()` y confirma ausencia de duplicados y orden estable. Después contrasta `ObtenerResumenes()` con una carga completa: el DTO contiene exactamente los campos consumidos por Application.

Errores comunes revisados

Error	Causa	Solución
Propiedad no proyectada	Se accede a una propiedad que no está en la proyección	Proyectar todas las propiedades necesarias
Tipo anónimo fuera de ámbito	Se intenta usar un tipo anónimo fuera del método	Proyectar a un DTO con nombre
Función no traducida	Se usa una función que EF Core no puede traducir	Usar funciones traducibles
Proyección antes del filtro	Se proyecta antes de filtrar	Filtrar y ordenar antes de proyectar
Tipo de colección incompatible	El DTO exige <code>List<T></code> pero la proyección entrega otra forma de secuencia	Usar <code>ToList()</code> cuando el <i>shape</i> de destino exige una lista; no convertirlo en una regla universal
Distinct no aplicado	Se olvidó el operador <code>Distinct</code>	Añadir <code>.Distinct()</code> antes de <code>ToList</code>

Analogía operativa

Las proyecciones con `Select` en una acería son como los resúmenes que el jefe de planta pide al archivo central. En lugar de recibir la carpeta completa de cada orden con todos sus documentos, el jefe pide solo los datos que necesita: el número de orden, el cliente y el estado. El archivo central prepara un resumen con esos datos y lo entrega. El jefe no necesita los documentos completos para tomar decisiones: solo los datos clave. Proyectar a un tipo anónimo es como pedir un resumen informal: se usa en el momento y no se archiva. Proyectar a un DTO es como pedir un formulario estandarizado: tiene nombre, se puede archivar y se puede usar en otros departamentos. Proyectar con `Distinct` es como pedir la lista de clientes únicos: sin duplicados. Proyectar con funciones de agregación es como pedir el total de planchas y el peso total: valores calculados en el archivo central. Así funcionan las proyecciones en EF Core: se seleccionan solo los datos necesarios, se reduce el volumen de información y se mejora el rendimiento.

Paso 10: Reto resuelto y conexión con el siguiente punto

Reto: Proyecta solo número de orden y cliente, y compara el `SELECT` con la carga de la entidad completa.

Cómo activar el reto en el checkpoint

Estos bloques ya existen comentados en el checkpoint. Actívalos sólo para reproducir esta variante y restaura después el estado original.

Archivo: src/AceriaData.Application/Interfaces.cs**Marcador:** RETO M03 3.3 - PUERTO SQL ENTIDAD COMPLETA

```
// Se activa coordinadamente con el repositorio y el caso de uso del reto.
string ObtenerSqlEntidadCompletaRetoProyeccion();
```

Archivo: src/AceriaData.Infrastructure/Repositories/Repositories.cs**Marcador:** RETO M03 3.3 - SQL ENTIDAD COMPLETA PARA COMPARAR SELECT

```
// Usa el mismo filtro y orden que ObtenerSqlProyeccion, pero devuelve el shape de entidad.
public string ObtenerSqlEntidadCompletaRetoProyeccion() => _context.OrdenesFabricacion
    .Where(o => o.Estado == "Pendiente")
    .OrderBy(o => o.FechaCreacion)
    .ToString();
```

Archivo: src/AceriaData.Application/ProyeccionesUseCase.cs**Marcador:** RETO M03 3.3 - PROYECCION MINIMA Y CLIENTES UNICOS

```
// Demuestra el Paso 10 y el laboratorio adicional: Distinct estable,
// DTO con nombre y comparación del SELECT mínimo frente a entidad completa.
var clientesReto = _unidad.Ordenes.ObtenerClientesUnicos();
var clientesEsperados = new[]
{
    "Constructora del Este",
    "Constructora del Norte",
    "Constructora del Sur"
};

if (!clientesReto.SequenceEqual(clientesEsperados))
    throw new InvalidOperationException("Reto 3.3: los clientes únicos no son estables o
    contienen duplicados.");

var resmenesReto = _unidad.Ordenes.ObtenerResmenes();
if (resmenesReto.Count != 5 ||
    resmenesReto.Any(r => string.IsNullOrEmpty(r.NumeroOrden) ||
        string.IsNullOrEmpty(r.Cliente) ||
        string.IsNullOrEmpty(r.Estado)))
    throw new InvalidOperationException("Reto 3.3: el DTO de resumen no contiene el shape
    esperado.");

var sqlMinimo = _unidad.Ordenes.ObtenerSqlProyeccion();
var sqlEntidad = _unidad.Ordenes.ObtenerSqlEntidadCompletaRetoProyeccion();

Console.WriteLine($"Reto 3.3 OK | Clientes: {string.Join(", ", clientesReto)} | DTOs:
{resmenesReto.Count}");
Console.WriteLine("SQL_MINIMO_INICIO");
Console.WriteLine(sqlMinimo);
Console.WriteLine("SQL_MINIMO_FIN");
Console.WriteLine("SQL_ENTIDAD_INICIO");
Console.WriteLine(sqlEntidad);
Console.WriteLine("SQL_ENTIDAD_FIN");
```

Comprobación esperada: comparación entre SELECT mínimo y carga de entidad completa.

La solución está representada por el estado acumulativo del propio checkpoint y sus métodos de repositorio. Tras validar **3.3 OK**, el siguiente estado parte exactamente de esta solución y añade **Proyecciones a DTOs**.